INTERVIEW GUIDE · SYSTEM DESIGN DOSSIER

Frontier — Portfolio Intelligence

A quant-grade portfolio manager & optimiser that explains itself in plain English. This guide documents the architecture, every major engineering trade-off, and the 20 hardest questions an interviewer will ask about it — with answers.

What's inside

- **Architecture** — a stateless scheduled ETL, a content-hashed static data API and a browser-side quant engine, with diagrams you can redraw on a whiteboard.
- **21 engineering trade-offs** — each with the rejected alternative and the cost accepted.
- **The algorithms** — FISTA with exact projections, Ledoit–Wolf, Bayes–Stein, Black–Litterman, HRP, risk parity, CVaR, HMM regimes, block bootstrap.
- **Quality evidence** — golden tests against scikit-learn, PyPortfolioOpt and CVXPY, and the real bugs they caught.
- **20 brutal interview questions** with model answers and follow-ups, a rapid-fire round and the numbers to memorise.

19,051
non-blank lines across TS · TSX · CSS · YAML · Python

113 tests
87 unit & golden (Vitest) + 26 end-to-end (Playwright)

$0 / month
GitHub Pages + Actions; no servers, no database

Live app: https://bhang-ux.github.io/frontier-portfolio/
Source: https://github.com/Bhang-ux/frontier-portfolio
Generated: 2026-11-04 · commit 8b22d2c · by `docs/interview-guide/build.mjs`

Contents

1 Executive summary & how to pitch it

2 System architecture

3 Engineering trade-offs — every major decision

4 The quant engine: algorithms you must be able to explain

5 Quality strategy: how the maths is proven right

6 20 brutal interview questions — with model answers

7 Rapid-fire round, numbers to memorise, honest limitations

How to use this guide

Read §1 until you can deliver the 30-second and 2-minute pitches without notes. Know the diagram in §2.1 well enough to redraw it on a whiteboard. For each trade-off in §3, be ready to argue *both* sides — interviewers push on the option you rejected. The answers in §6 are deliberately long: absorb the structure (claim → mechanism → evidence → limitation), then answer in your own words. Every number quoted is real and comes from this repository.

1 Executive summary & how to pitch it

Frontier is a portfolio manager and optimiser for complete beginners that is built like a professional quant system. It pulls live market data on a schedule, runs institutional-grade analytics (shrinkage covariance, efficient frontiers, risk parity, HRP, Black–Litterman, CVaR, Monte Carlo, regime detection) and translates every number into a plain-English sentence — with no servers, no database and no running cost.

The 30-second pitch

> "I built a free, always-on portfolio optimiser. A GitHub Actions cron job pulls prices for about a hundred US and Indian instruments every hour, analyses them with a TypeScript quant engine I wrote and verified against scikit-learn and CVXPY, and publishes static JSON to GitHub Pages. The browser runs the *same* engine in a Web Worker, so users can optimise and simulate their own portfolios instantly and privately — and every metric comes with a sentence a beginner understands."

The 2-minute version (structure)

1. **Problem.** Beginners get either toy calculators or professional tools they can't read. The hard part isn't the maths — it's estimation error and explanation.
2. **Architecture.** Stateless scheduled ETL → validation gate → content-hashed static API on a CDN → React SPA → Web Worker. Zero servers; the CDN is effectively the database and the cron is the backend.
3. **Quant core.** 4,725 lines, zero runtime dependencies: projected FISTA over exact constraint projections; Ledoit–Wolf and Bayes–Stein shrinkage; HRP; Black–Litterman; smoothed CVaR; Gaussian HMM; stationary bootstrap. Golden tests reproduce Python reference libraries to 1e-12.
4. **Beginner layer.** Risk questionnaire separating tolerance from capacity with hard safety caps; forecast-free model portfolios (risk budgeting within sleeves); deterministic natural-language explanations — no LLM, so explanations can never invent a number.
5. **Engineering quality.** 113 automated tests, CI that re-derives fixtures in Python, a validation gate that prevents bad data from ever shipping, post-deploy smoke tests, keepalive for the 60-day cron rule, CodeQL and Dependabot.

Key facts

Dimension	Value	Where it lives
Universe	98 fetched instruments + 2 constructed cash indices (US & India)	`pipeline/src/universe.ts`
Refresh cadence	Hourly at :17 Mon–Fri (UTC); every 4 h at :47 Sat–Sun	`.github/workflows/deploy.yml`
History depth	Daily since 2005-01-01, dividend & split adjusted	`pipeline/src/acquire.ts`

Payloads	manifest ~9 KB; snapshot ~86 KB gzip; ~19 KB gzip per instrument history	`packages/contracts`
Front-end	~220 KB gzip initial (JS+CSS), worker 24 KB, routes 4–10 KB each	`web/vite.config.ts`
Code size	quant 4,725 · pipeline+contracts 2,934 · web 9,791 · tests 1,223 lines	monorepo, npm workspaces
Optimiser speed	Max-Sharpe ≈ 9 ms for 6 assets; 20-year quarterly walk-forward ≈ 1 s	`packages/quant/src/optimize`

2 System architecture

2.1 The big picture

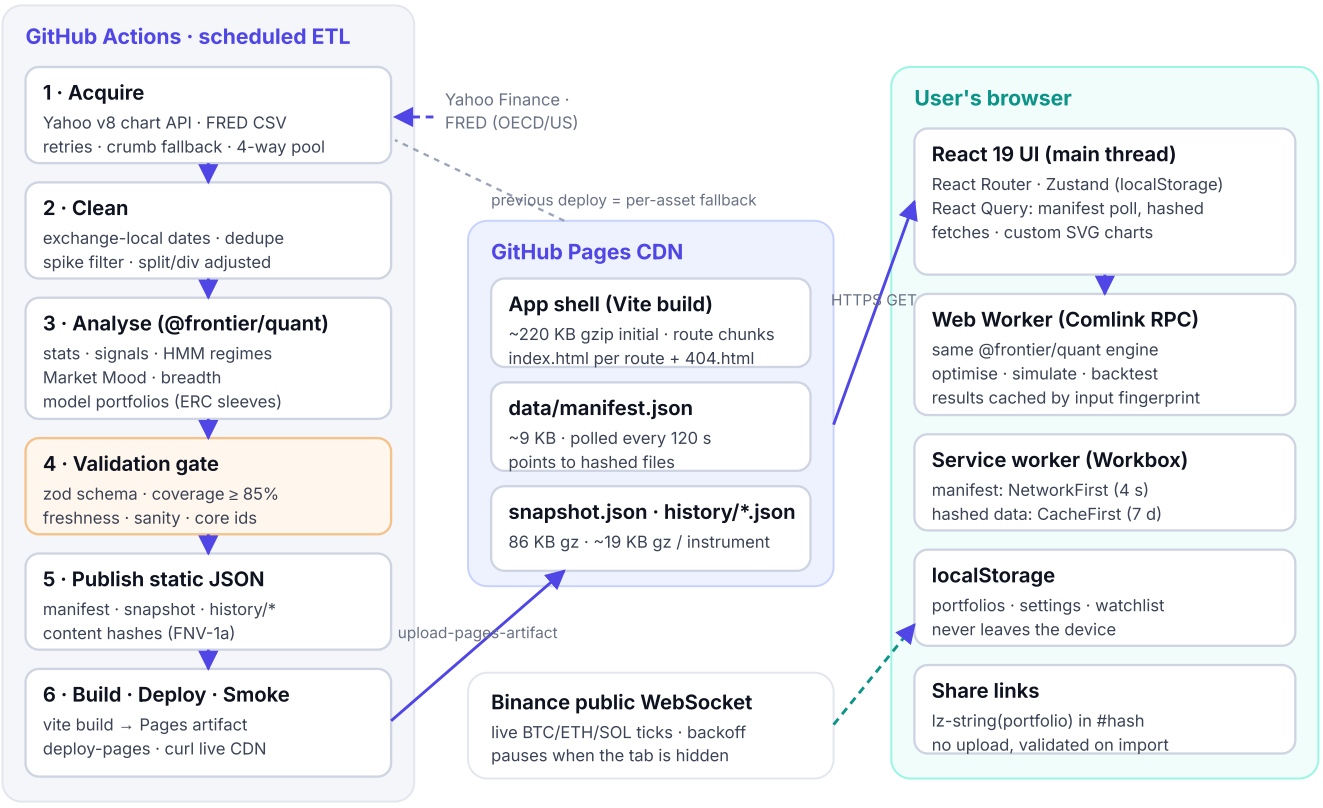


Figure 1 — Three runtimes, one engine. The scheduled job and the browser execute the same `@frontier/quant` package.

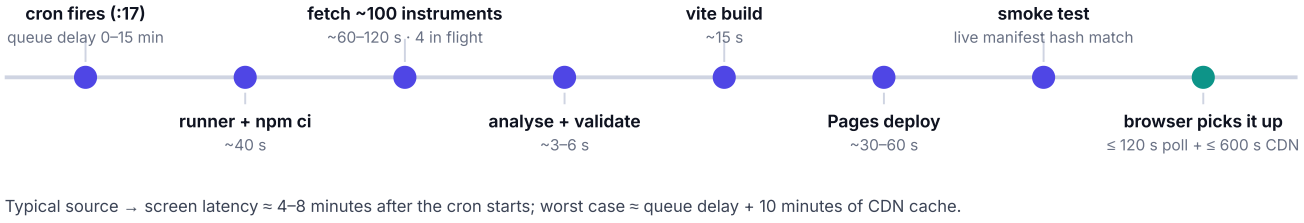
The design principle is **"move computation to where it is cheapest and safest"**. Anything shared by all users (fetching, cleaning, market-wide statistics, regimes, model portfolios) is computed once per hour in CI and published as static files. Anything specific to one user (their portfolio's risk, optimisation, simulations) is computed on their own device, which keeps their data private and makes the system's cost independent of the number of users.

2.2 Repository & component inventory

Package	Responsibility	Key dependencies
`packages/quant`	Numerical engine: linear algebra, statistics, covariance & return estimators, optimisers, Monte Carlo, backtests, HMM, factors, stress tests, health score, natural-language explanations.	none (by design)
`packages/contracts`	TypeScript types for every published JSON document + the history codec and content hash. The single source of truth for the pipeline ⇄ web interface.	none

`pipeline`	Scheduled ETL: universe definition, HTTP layer (retry/backoff/limiter), Yahoo & FRED clients, previous-deploy fallback, analytics stage, validation gate, publisher.	zod, quant, contracts
`web`	React 19 SPA: app shell, nine pages, chart library, Comlink worker, state stores, live WebSocket feed, PWA config, post-build route generation.	React, React Router, React Query, Zustand, d3-scale/shape, Radix, motion
`e2e` , `tools`	Playwright journeys; Python reference generator for golden fixtures; README screenshot tool.	Playwright; NumPy, SciPy, scikit-learn, CVXPY, PyPortfolioOpt

2.3 Lifecycle of a refresh



Typical source → screen latency ≈ 4–8 minutes after the cron starts; worst case ≈ queue delay + 10 minutes of CDN cache.

Figure 2 — From cron trigger to pixels.

1. **Acquire** (`pipeline/src/acquire.ts`). For each of 98 symbols, the Yahoo v8 chart endpoint is called with `period1` = 2005-01-01 through a 4-slot concurrency limiter. Hosts alternate between query1/query2. 429 and 5xx responses are retried up to 4 times with exponential backoff and full jitter, honouring `Retry-After` . A 401/403 triggers a one-time cookie + crumb handshake (the yfinance technique). If a symbol still fails, its history is fetched from the *currently deployed site* instead; only then is it recorded as a failure. Risk-free rates come from `^IRX` (US) and FRED's OECD India 3-month rate, and constructed cash indices accrue those rates daily.
2. **Clean** (`sources/yahoo.ts`). Timestamps are converted to *exchange-local* calendar days using the response's `gmtoffset` ; nulls are dropped; duplicates resolve to the latest print; one-day spikes of more than 40 % that fully revert the next day are removed as bad prints.
3. **Analyse** (`pipeline/src/build/*`). Per asset: trailing returns on calendar windows, risk statistics on 1/3/5-year windows with the calendar's empirical periods-per-year, weekly-return betas, technical indicators and a composite trend score ranked within peer groups. Per market: a two-state HMM regime, a six-component Market Mood score, breadth and a headline sentence. Ten model portfolios (5 profiles × 2 markets) are built and backtested.
4. **Validate** (`pipeline/src/validate.ts`). zod schema; all core instruments present; ≥ 85 % coverage; SPY's last bar ≤ 6 days old; finite, positive prices; ≥ 6 model portfolios. Failure exits non-zero: **nothing is deployed and the last good site keeps serving**.
5. **Publish & deploy**. One JSON file per instrument plus snapshot, intraday and manifest; Vite builds the app; `actions/deploy-pages` publishes atomically; a smoke job polls the live manifest until its `generatedAt` equals the one just built and checks that deep links return 200.
6. **Consume**. Open tabs poll the manifest every 120 s; when the snapshot hash changes, React Query fetches the new snapshot and a toast announces fresh data. Unchanged histories are never re-downloaded.

2.4 The static data API

Document	Contents	Caching strategy
`data/manifest.json`	Schema version, generation time, synthetic flag, snapshot hash & size, per-instrument history hash/end date/points, pipeline stats (duration, sources, failures, run id, commit).	`fetch(…, {cache: 'no-cache'})` → revalidates every poll; service worker NetworkFirst with a 4 s timeout so the app still opens offline.
`data/snapshot.json?v=<hash>`	All instruments' quotes, statistics, signals, summaries and sparklines; markets (regimes, mood, breadth); model portfolios; FX; risk-free rates; movers.	Immutable per hash → React Query `staleTime: Infinity`; service worker CacheFirst.
`data/history/<id>.json?v=<hash>`	Delta-encoded epoch days + 6-significant-digit adjusted closes. Lazy-loaded only for instruments a page needs.	Same as snapshot; 30-minute in-memory GC.
`data/intraday.json`	5-minute bars for eight headline instruments.	Per hash.

Why a manifest?

It is the **atomic pointer** of the system. Because every other file is addressed by content hash through the manifest, a client can never combine a new snapshot with an old history file — the classic "torn read" of multi-file static publishing. It also makes polling cheap: one ~9 KB request, conditional on ETag, every two minutes.

2.5 Client architecture

- **Routing.** React Router with a basename; all pages are lazy chunks. GitHub Pages has no rewrite rules, so `web/scripts/postbuild.mjs` writes an `index.html` into a folder per route *and per asset* (deep links return real 200s) plus a `404.html` fallback.
- **State.** Zustand stores persisted to localStorage: settings (currency, Beginner/Pro mode, theme, estimation choices, risk profile, watchlist) and portfolios (weights + total value + currency). Share links encode a portfolio with lz-string into the URL fragment, which never reaches a server; imports are validated and capped.
- **Server state.** React Query for the manifest (polled), snapshot and histories (immutable), and *worker computations*: each worker call is a query keyed by a fingerprint of its inputs, which gives de-duplication, caching and "keep previous result while recomputing" for free.
- **Compute.** One module Web Worker exposes `analyze`, `optimize`, `walkForward`, `rebalanceStudy`, `monthlyReturns` and `simulate` via Comlink. Inputs are structured-cloned (≈100 k numbers for a 10-asset, 20-year portfolio — milliseconds). Results are decimated to ≤ 700 points per series before crossing back.
- **Charts.** A small custom SVG library on d3-scale/d3-shape: index-based time axes (no weekend gaps), crosshair tooltips listing every series, keyboard navigation, shaded regime bands, a table-view twin for every chart, and a colour palette validated for colour-vision deficiency in both themes.

- **Live layer.** Stocks refresh with the pipeline; crypto streams over Binance's public WebSocket (market-data host first, main host second), with exponential reconnect and pausing in background tabs.

2.6 Deployment & CI/CD

Workflow	Trigger	What it guarantees
`ci.yml`	every push & PR	typecheck (4 projects), ESLint, Prettier, Vitest with coverage; Python re-derivation of golden fixtures then TS golden tests; production build with synthetic data; Playwright on desktop and mobile; bundle-size report.
`deploy.yml`	cron, push to main, manual	live data → validation gate → build → Pages deploy → live smoke test. Concurrency group `pages` without cancellation so deployments never interleave.
`screenshots.yml`	monthly, manual	README images regenerated from *live* data and committed (GITHUB_TOKEN pushes don't retrigger workflows).
`maintenance.yml`	weekly	re-enables scheduled workflows via the API, defeating GitHub's 60-day inactivity auto-disable.
`codeql.yml`, Dependabot	push, weekly	static security analysis; grouped dependency updates for npm, Actions and pip.

2.7 Failure modes

Failure	Detection	Behaviour	User impact
One symbol 404/429s	HTTP layer	retry → previous-deploy copy → recorded in manifest	that asset's data ≤ 1 refresh stale
Yahoo blocks the runner IP	coverage < 85 %	validation fails, no deploy, job red	site keeps last good data; freshness pill turns amber
Yahoo changes its JSON	parser throws → failures → gate	same as above	same as above
Bad data slips (e.g. bad print)	spike filter; vol sanity warnings	point removed / warning logged	none
GitHub cron delayed or skipped	manifest age	next run catches up (stateless)	data older than an hour
Deploy succeeds, CDN stale	smoke test hash compare	warning after 3 min	≤ 10 min (Pages max-age)
Worker throws (infeasible constraints)	React Query error	readable message from the feasibility checker	explains how to fix the constraints
WebSocket blocked	onerror/onclose	backoff, host rotation	crypto shows last snapshot price

2.8 Security & privacy model

- **No backend, no secrets.** The deploy uses only the short-lived, job-scoped `GITHUB_TOKEN` with least-privilege permissions per workflow (`pages: write, id-token: write` for deploy; `contents: write` only for screenshots; `actions: write` only for keepalive).
- **Untrusted inputs.** Share links and third-party data are treated as data: decoded JSON is type-checked, sanitised, capped at 40 holdings and rendered through React (escaped); WebSocket frames are parsed defensively. There is no `dangerouslySetInnerHTML`.
- **Supply chain.** Lockfile, `npm ci`, pinned major versions of first-party GitHub Actions, CodeQL and Dependabot. Known gap: GitHub Pages cannot set response headers, so a CSP would have to be a `<meta>` tag (listed under limitations).
- **Privacy.** Portfolios never leave the device; there are no analytics or cookies. The only third-party connection is the optional crypto WebSocket.

3 Engineering trade-offs — every major decision

Interviewers rarely ask *what* you built; they ask *why not the alternative*. Each row states the decision, the main alternative, the argument for, and the honest cost.

Area	Chosen	Rejected alternative	Why	Cost we accepted
Hosting	GitHub Pages + Actions	Vercel/Netlify functions, Render/Fly containers, Cloudflare Workers + D1	Free forever, global CDN, no cold starts, scheduler + CI + hosting next to the code, nothing to patch.	No server-side compute per request; fixed `max-age=600`; cron is best-effort; public repo required on the free tier.
Data serving	Static, content-hashed JSON	REST/GraphQL API over Postgres	Reads are CDN hits; zero ops; perfect cache semantics via hashes; scales to any audience.	Data only as fresh as the last run; no per-user queries; payload design matters.
ETL model	Stateless full refresh each run	Incremental updates with a state store (cache/branch/DB)	Idempotent and self-healing; dividend re-adjustments handled implicitly; trivial to reason about.	~100 full-history requests per run; more upstream load than deltas.
Market data	Yahoo v8 (keyless) + FRED	Alpha Vantage, Polygon, Tiingo, EODHD (keys, quotas or cost)	Free, deep history, covers NSE and US with adjusted closes.	Unofficial and rate-limited; terms of use; mitigated by fallbacks and a gate, but a paid vendor is the production answer.
Numerics language	TypeScript everywhere	Python backend (NumPy/CVXPY) or Pyodide in the browser	One engine for pipeline and browser; ~40 KB vs ~10 MB runtime; no server; Python retained for verification.	No BLAS/SIMD; we must write and prove our own solvers; float64 only.
Optimiser	Projected FISTA with exact projections	Interior-point QP (OSQP/Clarabel via WASM), quadprog	Tiny, dependency-free, warm-startable along frontiers, handles box + disjoint group constraints exactly.	First-order accuracy (1e-9…1e-12 here) and backtracking; overlapping groups fall back to Dykstra.
Max Sharpe	Golden-section along the λ-path	Cornuejols–Tütüncü homogenised QP	Re-uses the single MV kernel; works unchanged with any constraint the projector supports.	Needs the unimodality argument; ~25 solves per call (9 ms).
Risk model default	Ledoit–Wolf (constant correlation)	Sample covariance	Optimal shrinkage intensity; well-conditioned even when T is close to n; stabilises weights.	Biased towards the average correlation; less reactive than EWMA.

Return model default	Bayes–Stein shrinkage	Raw historical means	Cuts the error-maximisation problem at its root; still data-driven.	Still backward-looking; shrinkage target is a modelling choice.
Beginner portfolios	Forecast-free risk budgeting by profile	Mean–variance optimised “model” portfolios	Robust, explainable, no reliance on noisy return forecasts; follows strategic-allocation practice.	Not “optimal” in-sample; sleeve composition is opinionated.
Regimes	Two-state Gaussian HMM, filtered probabilities	Volatility thresholds; smoothed HMM; Markov-switching GARCH	Statistically principled, interpretable states, no look-ahead in the live signal.	Gaussian emissions miss fat tails; state labels can flip without sorting (we sort by variance).
Multi-market alignment	Weekday calendar + forward fill; weekly returns for betas	Intersection of trading days; daily betas	Keeps all observations; weekly sampling removes asynchronous-close bias (NSE closes before NYSE opens).	Holiday zero-returns slightly understate volatility; documented.
Compute placement	Web Worker + Comlink + React Query	Main thread, or server endpoints	UI never janks; results cached by input fingerprint; private by construction.	Structured-clone cost; worker debugging is harder.
Charts	Custom SVG on d3-scale/d3-shape	Recharts, ECharts, Highcharts	~15 KB, total control of accessibility and design tokens, index time axes, table twins.	More code to own; no canvas path for huge series (we decimate instead).
State	Zustand + localStorage	Accounts with a backend	Zero friction, private, offline-capable.	No cross-device sync; storage can be cleared.
Explanations	Deterministic NLG templates	LLM-generated text	Free, instant, offline, testable, can never hallucinate numbers.	Less varied prose; templates must be maintained.
Styling	Tailwind v4 + CSS variable tokens	CSS-in-JS, component library	Theme switching is a variable flip; no runtime cost; validated palette in one place.	Long class lists; v4 `@utility` idioms are new.
Monorepo	npm workspaces, source-level imports	pnpm + Turborepo, published packages	No build step between packages; one lockfile; one test runner (Vitest projects).	No task caching; workspace TS resolution must be configured carefully.
Test data	Synthetic generator flagged in UI; recorded API fixtures	Committing real market data	Deterministic, licence-free, offline CI.	Synthetic data cannot catch real-world data quirks — the live gate covers that.

Cache busting	`?v=hash` query from manifest	Hashed filenames	Stable file paths (easy fallback fetch and debugging) with exact invalidation.	Relies on CDNs keying on query strings (Pages/Fastly does).
SPA routing on Pages	index.html per route + 404.html	Hash routing; 404 redirect trick	Clean URLs with real 200s for every known page.	Build step must enumerate routes (done from the snapshot for assets).

3.1 The three decisions that shaped everything

(a) "The cron is the backend, the CDN is the database."

The requirement was permanent 24/7 hosting at zero cost. Free compute tiers either sleep (cold starts of 30 s+ on container hosts), cap invocations, or require credit cards and secrets. Market data changes at most a few times an hour for this audience, so a **batch** architecture is a perfect fit: compute shared results once, serve them as files. The consequence is that every per-user computation must happen client-side — which pushed the quant engine into TypeScript and into a Web Worker.

(b) One numerical engine in two runtimes.

Using Python in CI and JavaScript in the browser would mean two implementations of every statistic, guaranteed to drift. A single zero-dependency TypeScript package means the Sharpe ratio on the Markets page is computed by exactly the code that computes it on the Portfolio page. The price is that we can't lean on LAPACK or CVXPY at runtime; we earned trust instead with golden tests against those libraries.

(c) Robust estimation over optimisation cleverness.

Mean–variance optimisation is notoriously an *error maximiser* (Michaud, 1989): it over-weights assets whose returns are over-estimated and risks under-estimated. Rather than chase a more sophisticated optimiser, Frontier spends its complexity on **inputs** — Ledoit–Wolf and denoised covariances, Bayes–Stein and Black–Litterman returns — on **robust methods** that ignore returns entirely (risk parity, HRP), and on **honest evaluation** (walk-forward tests that show users how often simple methods win).

4 The quant engine: algorithms you must be able to explain

4.1 One kernel for every mean–variance objective

Every MV-type objective reduces to the constrained problem

```
w(λ) = argmin_{w ∈ C}  ½·λ·wᵀΣw - μᵀw + γ‖w - w̃‖²
C    = { w : lᵢ ≤ wᵢ ≤ uᵢ,  Σwᵢ = 1 } ∩ { asset-class bounds }
```

- **Minimum variance**: λ → ∞ (μ ignored). **Max utility**: fixed λ. **Efficient frontier**: the path λ ↦ w(λ), traced on a log grid with warm starts.
- **Target return / target volatility**: bisection on log λ, because the frontier's return and volatility are monotone decreasing in λ. Unreachable targets clamp with a readable warning.
- **Max Sharpe**: the efficient frontier σ*(r) is convex in r (value function of a parametric convex program), so the Sharpe ratio (r − r_f)/σ*(r) is quasi-concave along the path → **unimodal in λ**. A 10-point grid brackets the peak; golden-section search refines it to a 1e-4 bracket in log λ.
- **Max diversification**: maximise $\Sigma w_i\sigma_i$ / σ_p — which is exactly the Sharpe ratio with μ := σ and r_f := 0, so it reuses the same search.

4.2 The solver and its projections

Monotone FISTA (Beck & Teboulle 2009) with backtracking on the descent-lemma condition; the Lipschitz constant is initialised exactly as L = λ·λ_max(Σ) + 2γ (power iteration). Convergence is $O(1/k^2)$; the monotone variant never accepts an increase of the objective, which matters for the non-quadratic CVaR objective.

Capped-simplex projection. KKT gives $w_i(\tau) = \text{clip}(v_i - \tau, l_i, u_i)$. The sum $\Sigma w_i(\tau)$ is monotone non-increasing in τ, so bisection on τ (100 iterations max) is globally convergent and exact to machine precision; a final residual is spread over free coordinates.

Disjoint group constraints (e.g. "≤ 60 % stocks"). Dykstra's alternating projections initially failed: inexact inner projections made FISTA run its full 20 000 iterations (4.2 s). The fix was an exact projection: for a fixed τ, each group's multiplier merely clamps its group sum into [lo_G, hi_G], so the total S(τ) = Σ_G median(lo_G, S_G(τ), hi_G) + Σ_free clip(v_i − τ) is monotone in τ. One outer bisection plus one per active group — milliseconds. Overlapping groups still use Dykstra.

4.3 Estimators

Estimator	Formula / idea	Verified against
Ledoit–Wolf (identity)	$\hat{\Sigma} = (1-\delta)S + \delta\cdot\mu I$ with δ = min(β, δ̃)/δ̃ from the sample's fourth moments	scikit-learn `ledoit_wolf` (exact to 1e-15)
Ledoit–Wolf (constant correlation)	Target F: own variances, average correlation r̄; κ = (π − ρ)/γ, δ = clip(κ/T, 0, 1)	PyPortfolioOpt (exact to 1e-15)

EWMA	weights λ^(T−1−t), λ = 0.94 (RiskMetrics), weighted demeaning	unit tests (PD, scale)
RMT denoising	eigenvalues below $\lambda_+ = \sigma^2(1+\sqrt{(n/T)})^2$ replaced by their mean, trace preserved, unit diagonal restored	unit tests (PD, unit diagonal)
Bayes–Stein	$\mu_BS = (1-w)m + w\cdot\mu_o$, $w = (n+2)/((n+2) + T\cdot d^T S^{-1} d)$, μ_o = GMV return	unit tests
Black–Litterman	$\mu_BL = \pi + \tau\Sigma P^T(P\tau\Sigma P^T + \Omega)^{-1}(Q - P\pi)$; Ω He–Litterman or Idzorek (α = (1−c)/c)	PyPortfolioOpt (1e-13)

4.4 Robust allocators

Risk parity solves Spinu's strictly convex problem $\min \tfrac{1}{2} y^T \Sigma y - \Sigma b_i \ln y_i$ by cyclical coordinate descent; each coordinate update is the positive root of a quadratic, $y_i = (-c_i + \sqrt{(c_i^2 + 4\Sigma_{ii} b_i)})/(2\Sigma_{ii})$, and $w = y/\Sigma y$. Our risk contributions match to 1e-10 — tighter than the Clarabel reference (8e-6), which is why the golden test asserts equal contributions rather than equality with the reference.

HRP: correlation distance $\sqrt{(\tfrac{1}{2}(1-\rho))}$, single-linkage clustering via Prim's MST and union-find labelling that reproduces SciPy's linkage IDs exactly, leaf ordering by DFS, then recursive bisection with inverse-variance cluster risk. No matrix inversion, so it never explodes on near-duplicate assets.

Min CVaR (Rockafellar–Uryasev): $\mathrm{CVaR}_\alpha(w) = \min_\zeta \zeta + (1/((1-\alpha)T))\Sigma \max(\ell_t - \zeta, 0)$. The hinge is smoothed with an annealed soft-plus; ζ is minimised *exactly* inside every evaluation by a monotone root find, and Danskin's theorem gives the reduced gradient — removing a severe conditioning problem between ζ and w. Result: within 1 % of the HiGHS LP optimum.

4.5 Simulation, regimes and signals

- **Monte Carlo**: month-major simulation over Float64Arrays; exact lognormal moment matching ($E[1+r] = 1 + \mu/12$, $\mathrm{Var} = \sigma^2/12$); Student-t shocks applied to *simple* returns (a t-distribution has no moment-generating function); Politis–Romano stationary block bootstrap (mean block 6 months) of the portfolio's own history; SIP step-ups, fees, inflation-indexed withdrawals, depletion and goal-time statistics.
- **Gaussian HMM**: Baum–Welch with Rabiner scaling, four seeded restarts, states sorted by variance, Viterbi path for “regime since”, and **filtered** probabilities $P(s_t \mid x_{1\ldots t})$ for the live signal — smoothed probabilities would leak the future.
- **Trend score**: 40 % trend (price vs 50/200-day averages), 30 % 12-1 momentum percentile within a peer group, 20 % 1-year Sharpe percentile, 10 % overheating penalty from RSI — each point traceable to a sentence.

5 Quality strategy: how the maths is proven right

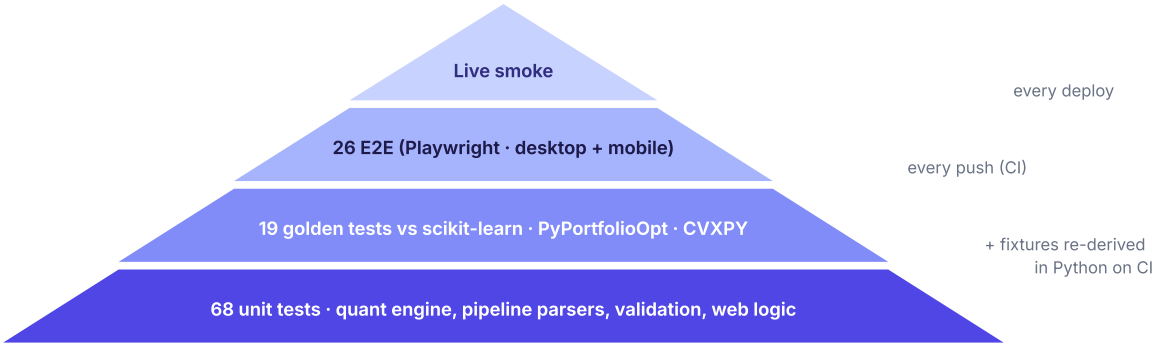


Figure 3 — 113 automated checks plus a live smoke test after every deployment.

5.1 Golden tests

`tools/reference/generate_fixtures.py` draws daily returns from a seeded two-factor model (600 days × 8 assets) and computes reference results with independent, widely used libraries. The fixture stores the *inputs too*, so the TypeScript tests consume byte-identical data.

Component	Reference	Tolerance & rationale
Sample covariance, Ledoit–Wolf ×2	NumPy, scikit-learn, PyPortfolioOpt	1e-15 — deterministic closed forms
HRP weights + linkage matrix	PyPortfolioOpt / SciPy	1e-12 and identical cluster IDs
Black–Litterman (both Ω)	PyPortfolioOpt	1e-13
Min variance, utility, max Sharpe, targets, groups, max diversification	CVXPY + Clarabel	objective ≥ reference − 1e-6; weights 2e-4…2e-3 (interior-point solvers stop at ~1e-8 gaps)
Min CVaR	CVXPY + HiGHS LP	within 1 % (smoothing)
Metrics, OLS, eigenvalues, normal quantiles	pandas, NumPy LAPACK, SciPy	1e-10…1e-14

5.2 Bugs the tests actually caught

1. **Group constraints hung.** Dykstra-in-FISTA needed 4.2 s and never met tolerance → exact disjoint-group projection (milliseconds).
2. **OLS was biased by 2e-9.** A “harmless” default ridge of 1e-12·trace distorted small-scale regressors → removed; collinearity handled by Cholesky jitter only.
3. **Monte Carlo drifted high.** The lognormal drift made E[1+r] = e^(μ/12) instead of 1 + μ/12 (≈ 0.4 %/yr error) → exact moment matching; zero-volatility test now equals the closed-form SIP value to the cent.
4. **“Failure” that was a success.** ERC weights differed from Clarabel by 5e-6; measuring risk contributions showed ours were 1000× more equal → the test now asserts the defining property.
5. **A month-offset bug** (`Date.UTC` takes 0-based months) in the fetch window was caught in review while fixing a no-op expression.

5.3 Pipeline & front-end tests

- Parsers run on a recorded-shape Yahoo response with nulls, duplicate bars and IST timestamps; FRED's “.” missing values; the history codec round-trip; universe integrity (unique, file-safe IDs; valid benchmarks and proxies); the concurrency limiter; and an end-to-end offline pipeline run through the validation gate.
- Playwright drives the production build: every route on desktop and a Pixel 7 profile with console-error assertions; the full questionnaire → adopt journey; the asset picker; the optimiser; the command palette; theme switching; share-link import.
- **Accessibility by construction**: categorical palette validated for adjacent-pair CVD separation (OKLab ΔE ≥ 8) and contrast in both themes; text tokens ≥ 4.5:1 (WCAG AA); signal and status colours always paired with an icon and a label; every chart has a keyboard crosshair and a table view.

6 20 brutal interview questions — with model answers

Each question lists what the interviewer is really probing, a model answer grounded in this codebase, and the follow-up you should expect.

Q1 Walk me through everything that happens between Yahoo publishing a new price and a user seeing it in Frontier. Where is the latency, and how would you get it under one minute?

What they're testing: End-to-end systems thinking, knowing your own critical path, and whether you can reason about trade-offs instead of just adding infrastructure.

The path. A GitHub Actions cron (`17 * * * 1-5`) queues a run. The runner installs dependencies (~40 s), the pipeline fetches ~98 symbols through a 4-slot limiter (~1–2 min), analyses and validates (~5 s), Vite builds (~15 s), `deploy-pages` publishes atomically (~30–60 s), and a smoke job confirms the live manifest's `generatedAt` . In the browser, React Query polls `manifest.json` every 120 s with `cache: 'no-cache'` ; when the snapshot hash changes it fetches `snapshot.json?v=<hash>` and a toast announces fresh data.

Latency budget. (1) Scheduling: up to an hour between runs plus GitHub's queue delay at busy minutes — the dominant term, which is why the cron uses minute 17, not 0. (2) Run time: ~3–5 minutes. (3) CDN: GitHub Pages sends `max-age=600` , so an edge may serve the old manifest for up to 10 minutes. (4) Polling: ≤ 2 minutes. Crypto bypasses all of this through the Binance WebSocket and is real-time.

Getting under a minute requires changing the architecture, not tuning it: (a) move the live quote layer to a push channel — a small edge function (Cloudflare Worker) proxying a licensed real-time feed with server-sent events, while the heavy daily analytics stay in the batch job; (b) or have the pipeline run continuously (a free-tier VM or a self-hosted runner) publishing only a tiny `quotes.json` every 30 s to an object store with short TTLs. I deliberately didn't: the users are long-horizon investors for whom hourly is plenty, and the zero-cost, zero-ops property was a hard requirement.

Likely follow-up: "GitHub cron can be delayed or skipped entirely — how would you know?" → manifest age is shown in the UI (freshness pill turns amber after 3 h), every run writes a job summary, and an external uptime monitor on the manifest age would alert.

Q2 Your entire backend is a cron job scraping an unofficial API. What happens the day Yahoo starts returning 429s from GitHub runners, or changes its JSON? How do you find out, and what does the user see?

What they're testing: Failure-mode design, graceful degradation, observability — and honesty about vendor risk.

Layer 1 — the HTTP client (`pipeline/src/sources/http.ts`): 429/5xx are retried four times with exponential backoff and full jitter, honouring `Retry-After` ; 4xx are fatal except 401/403, which

trigger a one-time cookie + crumb handshake (the technique yfinance uses when Yahoo tightens access). Requests alternate between query1/query2 hosts and run at most four in parallel.

Layer 2 — per-instrument fallback: if a symbol still fails, the pipeline downloads that instrument's history from the currently deployed site ("the CDN is the backup database") and marks its source as `previous-deploy`. One flaky symbol never regresses the site.

Layer 3 — the validation gate: if Yahoo blocks broadly, coverage drops below 85 % or a core instrument (SPY, Nifty, USD/INR…) is missing; if the JSON shape changes, the parser throws for every symbol and the same gate trips. The job exits non-zero *before* the deploy step, so GitHub Pages keeps serving the last good build.

Observability: failures are emitted as workflow annotations and a step summary; the manifest records sources and failures per run; the UI's freshness pill shows the data's age. **User impact**: data becomes stale but never wrong or broken.

Honest answer on vendor risk: Yahoo's endpoints are unofficial and its terms restrict automated use, so for anything beyond a portfolio project I would put a licensed vendor (Polygon, Tiingo, EODHD) behind the same `sources/` interface — the contracts package means the rest of the system wouldn't change.

Likely follow-up: "What if Yahoo returns plausible but wrong numbers?" → spike filter for one-day bad prints, volatility sanity warnings, validation on finite positive prices; next step would be cross-checking a second source on core instruments.

Q3 Why is mean–variance optimisation called an "error maximiser", and what specifically did you do about it?

What they're testing: Whether you understand estimation risk — the central problem of practical portfolio construction — beyond textbook Markowitz.

The optimiser treats μ and Σ as truth. Estimation errors are not random noise in the output: the optimiser systematically *seeks out* assets whose expected return is over-estimated and whose risk is under-estimated, because those look best (Michaud 1989). Errors in μ dominate — Chopra & Ziemba (1993) estimate they matter about ten times more than errors in variances. With 10 years of data you can barely distinguish a 6 % asset from a 10 % one (standard error of a mean ≈ σ/√T ≈ 16 %/√10 ≈ 5 %).

Frontier attacks this at four levels:

1. **Better Σ**: Ledoit–Wolf shrinkage to a constant-correlation target by default (optimal intensity, always well-conditioned), with EWMA and Marchenko–Pastur denoising as options.
2. **Better μ**: Bayes–Stein shrinkage towards the minimum-variance portfolio's return by default; CAPM and equilibrium priors; Black–Litterman to blend views with explicit confidence.
3. **Constraints as regularisation**: per-asset caps (default 40 %), minimums and asset-class caps; optional L2 penalty towards equal weights. Jagannathan & Ma (2003) show that caps act like covariance shrinkage.
4. **Methods that don't need μ**: risk parity and HRP, and a walk-forward test that shows users, out of sample and after costs, how often these beat "optimal" portfolios — the DeMiguel, Garlappi & Uppal (2009) result made tangible.

And for beginners, the recommended model portfolios use **no return forecasts at all**: allocation by risk profile, equal risk contribution inside each sleeve.

Likely follow-up: "Then why offer max-Sharpe at all?" → it's the reference point everyone knows; shown with shrunk inputs, guard-rails and an out-of-sample test beside it.

Q4 Explain Ledoit–Wolf shrinkage. Why the constant-correlation target rather than the identity, and how did you prove your implementation is correct?

What they're testing: Depth on one statistical technique, and verification discipline for numerical code.

The sample covariance S is unbiased but noisy: with n assets it has n(n+1)/2 parameters, and its smallest eigenvalues are biased towards zero, which an optimiser exploits. Ledoit–Wolf replaces S with $\hat{\Sigma} = \delta F + (1 - \delta)S$, a convex combination with a structured, low-variance target F. The key result is a **consistent closed-form estimate of the δ that minimises expected Frobenius loss**: $\delta^* = \kappa/T$ with $\kappa = (\pi - \rho)/\gamma$, where π is the sum of asymptotic variances of the entries of S, ρ the corresponding covariances with F, and $\gamma = \|F - S\|^2$ the target's misspecification.

Target choice: the identity target (Ledoit–Wolf 2004a) shrinks every variance towards the average variance — fine for equally scaled signals, but it distorts assets whose volatilities differ by 10× (Treasury bills vs Bitcoin). The constant-correlation target (2004b) keeps each asset's own variance and shrinks only the correlations towards their average, which is the structure that actually matters for equities. Frontier offers both; constant-correlation is the default.

Proof of correctness: `ledoitWolfIdentity` mirrors scikit-learn's estimator, and `ledoitWolfConstantCorrelation` mirrors PyPortfolioOpt's — including their conventions (ddof = 0 vs 1). Golden tests run both against the Python libraries on a seeded 600 × 8 factor-model sample and match the shrinkage intensity to 12 decimals and every matrix entry to 1e-15. CI re-generates the fixtures in Python on every push, so an upstream library change would surface immediately.

Likely follow-up: "When does shrinkage hurt?" → when the true structure differs strongly from the target (e.g. distinct sector blocks); then a factor-model or block target, or RMT denoising, is better.

Q5 You maximise the Sharpe ratio, which is non-convex in the weights. How do you know you found the global optimum — with box and asset-class constraints?

What they're testing: Mathematical maturity: convexity arguments, problem transformations, and knowing when a method is actually correct.

There are two standard routes. The textbook one is the **Cornuejols–Tütüncü homogenisation**: minimise $y^\top \Sigma y$ subject to $(\mu - r_f)^\top y = 1$ and $y \geq 0$, then $w = y/\Sigma y$; box constraints become $y \leq \kappa u$ with an extra variable κ. It's convex, but it needs a different constraint set from every other objective — I use it in the Python reference to generate the golden answer.

Frontier's engine uses a route that reuses its single mean–variance kernel. Let σ*(r) be the minimum volatility achievable at return r over the constraint set C. Because C is convex, the value function of the parametric problem $\min\{w^\top \Sigma w : \mu^\top w = r, w \in C\}$ is convex in r, so σ*(r) — the frontier — is convex. The Sharpe ratio along the frontier, $(r - r_f)/\sigma^*(r)$, is a ratio of an affine function to a positive convex function, hence **quasi-concave**: it has no local maxima other than

the global one. The frontier is parameterised monotonically by the risk-aversion λ, so Sharpe is unimodal in log λ.

The implementation brackets the peak with a 10-point log-spaced λ grid (warm-started FISTA solves) and refines with golden-section search to a 1e-4 bracket. Any interior frontier point is efficient, so the tangency portfolio lies on this path even with asset-class constraints. The golden test checks the result's Sharpe is ≥ CVXPY's minus 1e-6, and a property test checks it beats every one of 60 frontier points and 2,000 random feasible portfolios. Degenerate case: if no feasible portfolio beats the risk-free rate, Sharpe has no meaningful maximum; the optimiser returns minimum variance with a plain-English warning.

Likely follow-up: "Why not maximise Sharpe directly by projected gradient ascent?" → also valid (it's pseudo-concave where the numerator is positive), but slower near boundaries and not reusable for target-return/volatility.

Q6 Walk me through your projected-gradient solver. Why FISTA, why the monotone variant, how do you choose the step size, and what broke when you added asset-class constraints?

What they're testing: Optimisation fundamentals, and the ability to debug numerical performance problems.

The problems are smooth (quadratic or smoothed), small (n ≤ ~100), and the feasible set has a cheap exact projection — the ideal setting for accelerated projected gradient. FISTA achieves $O(1/k^2)$ versus $O(1/k)$ for plain projected gradient. I use Beck & Teboulle's **monotone** variant, which never accepts a step that increases the objective; plain FISTA can oscillate, which matters for the soft-plus CVaR objective. Step size 1/L comes from the exact Lipschitz constant $L = \lambda \cdot \lambda_{max}(\Sigma) + 2\gamma$ (λ_{max} by power iteration) with backtracking on the descent-lemma inequality as a safety net, and a gentle $L \leftarrow 0.95L$ per iteration so one conservative backtrack doesn't slow everything down.

The projection onto $\{l \le w \le u, \Sigma w = 1\}$ is exact: $w_i = \text{clip}(v_i - \tau, l_i, u_i)$ with τ found by bisection, because $\Sigma w_i(\tau)$ is monotone.

What broke: for constraints like "stocks ≤ 35 %", I first combined the box-simplex projection with half-space projections using Dykstra's algorithm. The golden test passed — but took **4.2 seconds**. Profiling showed FISTA hitting its 20,000-iteration cap: Dykstra's inner iterations converge only linearly, and the slightly inexact projections prevented the outer stopping criterion (1e-12) from ever being met. The fix was mathematical, not a tolerance hack: for *disjoint* groups the KKT conditions decouple. At a fixed τ, each group's multiplier just clamps its group sum into [lo, hi], so the total $\Sigma_G \text{median}(lo_G, S_G(\tau), hi_G) + \Sigma_{free}\,\text{clip}(\cdot)$ is monotone in τ — one outer bisection plus one bisection per active group gives the exact projection. Runtime dropped to milliseconds. Overlapping groups still use Dykstra, which is correct but slower.

Likely follow-up: "Why not an interior-point method?" → more accurate per iteration, but needs linear-system solves, has no cheap warm starts along a frontier, and would be a large dependency; the golden tests show first-order accuracy is more than sufficient.

Q7 HRP, risk parity and mean–variance: when would you choose each? What did your walk-forward test show, and why does that test matter?

What they're testing: Practical portfolio-construction judgement and an understanding of in-sample vs out-of-sample evaluation.

Mean–variance is right when you have credible return views and want to express them precisely — e.g. via Black–Litterman with explicit confidence. It is fragile without them. **Equal risk contribution** needs only covariances: every holding contributes the same amount of volatility; robust and intuitive, but it ignores clustering — five tech stocks each get an equal risk budget as if independent. **HRP** first clusters assets by correlation distance, orders them so similar assets are adjacent, then splits risk recursively between halves. Similar assets share a budget, it never inverts Σ (so near-duplicates don't explode), and it tends to be stable out of sample (López de Prado 2016).

Why the walk-forward test matters: any optimiser looks perfect on the data it was fitted to. `walkForward()` re-estimates every month or quarter using only the trailing 252 trading days, chooses weights, then holds them (letting them drift) until the next rebalance, charging transaction costs on turnover. That is the only fair comparison. On typical multi-asset universes, equal weight, risk parity and HRP land close to or ahead of max-Sharpe on risk-adjusted returns, with far lower turnover — max-Sharpe chases last year's winners. Because the test runs on the user's own holdings, the conclusion is shown to them rather than asserted.

Likely follow-up: "HRP's bisection splits the ordered list in halves rather than following the dendrogram — is that a flaw?" → yes, a known critique; HERC (Raffinot 2018) follows the tree. I kept the original to match the reference implementation exactly.

Q8 Your app says "the market is in a turbulent regime with 80 % probability". What exactly does that number mean, and how could it be misleading?

What they're testing: Statistical rigour, look-ahead bias, and communicating uncertainty.

It comes from a two-state Gaussian Hidden Markov Model fitted to ten years of daily log returns (×100 for numerical scale) of the S&P 500 or Nifty 50. Each hidden state has its own mean and variance; a 2×2 Markov matrix governs switching. Parameters are fitted by Baum–Welch (EM) with Rabiner scaling from four seeded random starts, keeping the best likelihood; states are then **sorted by variance** so "state 0 = calm" is stable across runs (EM is invariant to label permutations).

The 80 % is the **filtered** probability $P(s_T = \text{turbulent} \mid x_1 \ldots x_T)$ from the scaled forward pass — what an investor could have known that day. I deliberately don't use the smoothed probabilities $P(s_t \mid x_1 \ldots x_T)$, which look cleaner historically but use future data: a backtest built on them is look-ahead biased. The chart of past regimes on the Today page shows filtered values for the same reason.

Ways it misleads: (1) it is a probability *under the model*, and Gaussian emissions understate fat tails, so the turbulent state partly absorbs crashes; (2) parameters are re-estimated every run, so history can shift slightly; (3) regime persistence is not a forecast of returns; (4) a two-state model is a simplification — three states (bull/bear/crisis) fit better but are less stable. The UI words it as

"most likely" or "probably" depending on the probability and explains what the regime implies behaviourally (don't panic-sell; consider rebalancing), not as a trading signal.

Likely follow-up: "How would you validate it?" → out-of-sample log-likelihood vs a single-Gaussian model, forecast evaluation of realised volatility conditional on the filtered state, and synthetic-data recovery tests (one exists in the suite).

Q9 You mix NSE stocks, NYSE ETFs, 24/7 crypto and USD/INR. How do you align them for portfolio returns, correlations and betas without introducing bias?

What they're testing: Real-world data-engineering skill in finance: calendars, time zones, currencies and asynchronous trading.

Dates: every bar is converted to an *exchange-local* calendar day using the response's GMT offset (a US bar stamped 13:30 UTC and an Indian bar at 03:45 UTC both map to their local trading day). Dates are integer epoch days throughout — timezone-free and cheap to compare.

Calendar: `alignSeries` takes the union of observation dates restricted to Monday–Friday, forward-fills holidays, and starts when the youngest instrument begins so every column is real data. Crypto weekend moves roll into Monday's return. Annualisation uses the *empirical* periods per year of the aligned calendar (≈ 260 for the weekday union, 252 for US stocks alone, 365 for crypto alone) rather than a hard-coded 252.

Currency: every series is converted to the portfolio's base currency with the forward-filled USD/INR rate before computing returns, so an Indian investor's S&P 500 position includes its currency return — which is real risk for them.

Asynchronous trading: India closes before New York opens, so a global shock shows up in Nifty one calendar day after the S&P 500. Daily correlations and betas are biased towards zero. Frontier computes betas and hypothetical-shock sensitivities on **weekly** returns, where the one-day offset is diluted.

Known residual bias: forward-filled holidays create zero returns that slightly understate volatility in multi-market portfolios; a holiday-aware intersection calendar or Newey–West-style adjustments would remove it. It's documented on the Learn page.

Likely follow-up: "Dimson betas?" → regressing on lead and lag market returns is the classical fix for non-synchronous trading; weekly sampling is the simpler alternative I chose.

Q10 Your Monte Carlo planner offers a bootstrap, a normal model and a fat-tailed model. Why the block bootstrap, what does the block length trade off — and what bug did you find in the parametric models?

What they're testing: Simulation design and the subtle statistics of compounding.

An i.i.d. normal model ignores the three stylised facts that hurt investors most: fat tails, negative skew and volatility clustering. The **stationary block bootstrap** (Politis & Romano 1994) resamples the portfolio's own historical monthly returns in blocks with geometrically distributed lengths (mean 6 months): at each step a path continues its current block with probability 5/6 or jumps to a

random month. Blocks preserve short-range autocorrelation and clusters like 2008's consecutive losses; the random lengths keep the resampled series stationary. Short blocks lose dependence; long blocks reduce diversity because there are only a few hundred months to sample from — 6 months is a common compromise for monthly data.

The bug: the first GBM implementation used a log-drift of $\mu/12 - \tfrac{1}{2}\sigma^2/12$ and exponentiated. That makes $E[1 + r] = e^{(\mu/12)}$, not $1 + \mu/12$ — an upward bias of about 0.4 % a year at $\mu = 10$ %, which compounds over 30 years. A zero-volatility unit test caught it because the simulated value didn't equal the closed-form SIP future value. The fix is exact lognormal moment matching: $s^2 = \ln(1 + (\sigma^2/12)/(1 + \mu/12)^2)$ and $m = \ln(1 + \mu/12) - s^2/2$. For the Student-t model I had been exponentiating t-distributed shocks too — but a Student-t has **no moment-generating function**, so $E[e^{(sT)}]$ is infinite and the sample mean is dominated by rare extreme draws. The fat-tailed model now shocks simple returns directly (floored at −95 %).

Implementation detail: simulation is month-major over Float64Arrays (state per path), giving exact percentiles every month; 6,000 paths × 30 years runs in well under a second in the worker.

Likely follow-up: "Isn't history too short for a 30-year bootstrap?" → yes; that's why the parametric models exist, and the Advisor's forward return is shrunk halfway to a Sharpe-0.3 prior.

Q11 What is the difference between VaR and CVaR? Why did regulators move to expected shortfall, and how does Frontier compute and use each?

What they're testing: Risk-measure literacy: coherence, tails and estimation.

VaR at 95 % is a quantile: the loss exceeded on only 5 % of days. It says nothing about *how bad* those days are, and it is not sub-additive — merging two portfolios can show more VaR than the sum, which punishes diversification. **CVaR / expected shortfall** is the average loss in that tail. It is a coherent risk measure (Artzner et al. 1999) and it is what Basel's Fundamental Review of the Trading Book adopted (97.5 % ES) for exactly these reasons. It is also convex in the weights, so it can be *optimised* as a linear programme (Rockafellar & Uryasev 2000).

Frontier reports, per day: historical VaR (empirical 5 % quantile with linear interpolation, matching NumPy), historical CVaR (mean of losses at or beyond that quantile), parametric VaR, and Cornish–Fisher VaR, which adjusts the normal quantile for skewness and excess kurtosis — on negatively skewed data it is correctly larger than the Gaussian VaR (a unit test asserts this). The Portfolio page translates VaR and CVaR into money ("on a bad day expect to lose at least ₹4,800…"). The **min-CVaR objective** uses the Rockafellar–Uryasev formulation with soft-plus smoothing and an exact inner minimisation over ζ; it lands within 1 % of the HiGHS LP optimum. The health score's crash-resilience component uses maximum drawdown, which beginners understand better than tail averages.

Likely follow-up: "How stable is a 95 % CVaR from three years of daily data?" → only ~38 tail observations; the estimate is noisy, which is why it is shown with context and not used as the only risk measure.

Q12 How does the client know there is new data — and how do you guarantee it never mixes an old snapshot with new history files, given CDN and service-worker

caching?

What they're testing: Caching, consistency and the subtle failure modes of static publishing.

There are three caches between the user and the files: GitHub Pages' Fastly CDN (fixed `max-age=600`), the browser HTTP cache, and a Workbox service worker. Naively polling `snapshot.json` and `history/X.json` invites **torn reads**: a fresh snapshot with a stale history from a different run.

The fix is a single mutable pointer. Only `manifest.json` is fetched by plain URL — with `cache: 'no-cache'` (conditional revalidation every poll) and a NetworkFirst service-worker rule with a 4-second timeout so the app still opens offline. The manifest lists a content hash (FNV-1a of the exact bytes) for the snapshot and every history file. All other requests carry `?v=<hash>`, which makes each URL immutable: React Query caches them with `staleTime: Infinity`, and the service worker uses CacheFirst. A client reading manifest version N can only ever request files from version N. History files whose bytes didn't change keep their hash and are not re-downloaded.

Deploys are atomic at the Pages level (one artifact), and the post-deploy smoke test waits until the live manifest's `generatedAt` equals the build's before marking the run green. Residual window: an edge can serve the previous manifest for up to 10 minutes — consistent, merely older.

Likely follow-up: "Why query-string hashes instead of hashed filenames?" → stable paths make the previous-deploy fallback and debugging trivial; Fastly keys on the full URL, so invalidation is still exact.

Q13 Why run the analytics in a Web Worker? How does data cross the boundary, what does that cost, and how do you avoid recomputing?

What they're testing: Front-end performance engineering beyond the framework.

A max-Sharpe solve takes ~9 ms, but a frontier with 40 points, 1,500 random portfolios, five special portfolios and an analysis with stress tests and factor regressions adds up to hundreds of milliseconds, and a 20-year walk-forward about a second. On the main thread that freezes typing in the holdings editor. The worker keeps the UI at 60 fps regardless.

Boundary: Comlink wraps `postMessage` in typed async RPC. Inputs are structured-cloned — a 10-asset, 20-year request is ~100 k numbers, which clones in a few milliseconds; outputs are decimated to ≤ 700 points per series before crossing back. Transferable ArrayBuffers would avoid copies, but at these sizes the gain doesn't justify the API complexity.

Avoiding recomputation: every worker call is a React Query query whose key is a *fingerprint* of the request — ids, weights and settings verbatim, but series summarised by length, first and last date and last value, so keys stay small and computing them is cheap. Identical requests are de-duplicated and cached; `keepPreviousData` shows the previous result at reduced opacity while a new one computes; holdings input is debounced by 350 ms. Derived arrays are memoised so the request object only changes when its inputs do.

Likely follow-up: "Would WebAssembly help?" → for n in the hundreds or Monte Carlo with millions of paths, yes (SIMD); for current sizes JS JIT is within ~2× and keeps one codebase.

Q14 A senior engineer says: “Numerical code belongs in Python. Why didn't you use FastAPI with NumPy, CVXPY and a database?” Defend your choice.

What they're testing: Architectural reasoning under pushback; knowing the costs of your own choice.

I'd agree for a research desk. For this product the constraints point the other way:

- **Cost & availability**: a Python API needs an always-on host. Free tiers sleep (30-second cold starts) or expire; the requirement was permanent, free hosting. A static site plus a batch job has no idle cost and no cold start.
- **Privacy**: portfolio analysis on the server means sending holdings to it. In the browser, portfolios never leave the device — with no accounts, no GDPR surface and no breach risk.
- **Scalability**: per-user compute happens on users' devices; serving a million users costs the same as serving one.
- **One implementation**: the pipeline needs the same statistics the browser shows; TypeScript lets both use one engine. Pyodide would ship a ~10 MB runtime to every visitor.

The costs are real: no BLAS (fine at n ≤ 100, where a 100×100 Jacobi decomposition takes milliseconds), no off-the-shelf convex solver (hence a custom FISTA with exact projections), and the burden of proof falls on me. That is why Python stays in the repository as the **oracle**: CI regenerates golden fixtures with scikit-learn, PyPortfolioOpt and CVXPY and the TypeScript engine must match them. The architecture would change if requirements did — e.g. thousands of assets or proprietary data would justify a server.

Likely follow-up: “What about float precision in JS?” → JS numbers are IEEE-754 doubles, the same as NumPy's float64; the differences are summation order and libm, covered by Neumaier-compensated sums where it matters and by test tolerances.

Q15 How would this scale to 10,000 instruments and a million daily users? What breaks first?

What they're testing: Scalability reasoning: identify the real bottlenecks rather than reciting “add Kubernetes”.

Users: almost nothing breaks — reads are static files on a CDN, and all per-user compute runs on the client. At a million users the first issue is GitHub Pages' soft bandwidth limit (~100 GB/month) and terms of service; the fix is moving the same artifact to a commercial CDN (Cloudflare Pages / R2 + CDN) with proper cache headers — a config change.

Instruments — this is where it breaks, in order: (1) **ingestion**: 10,000 full-history requests per hour would get the job blocked; switch to a licensed bulk/end-of-day feed, incremental updates with a state store, and fan-out across a job matrix. (2) **Snapshot size**: ~1 KB per instrument → 10 MB; shard it (per market/sector), keep a compact index for search, lazy-load detail. (3) **Client analytics** stay fine because a user's portfolio is still 5–50 assets, but universe-wide optimisation would not: an n = 10,000 covariance has 50 M entries. Use a factor model $\Sigma = BFB^{T} + D$ (O(nk)), which also improves estimation, and run screening server-side. (4) **Pipeline compute**: per-asset analytics are embarrassingly parallel; the HMM and model portfolios are per market. A matrix strategy or a small worker pool handles it.

Features that would force a backend: accounts with cross-device sync (an edge KV/SQLite like Cloudflare D1), alerts (push notifications need a server-side scheduler), or real-time quotes (a streaming proxy).

Likely follow-up: "How would you make ingestion incremental safely?" → fetch the last month, compare overlapping adjusted closes; any divergence beyond tolerance signals a dividend/split re-adjustment → refetch that symbol's full history.

Q16 Your app effectively tells beginners what to buy. What are the regulatory and ethical risks, and how does the design mitigate them?

What they're testing: Product judgement and responsibility — increasingly asked of engineers in fintech.

In India, giving personalised investment advice for consideration requires SEBI registration as an Investment Adviser; in the US the analogue is the Investment Advisers Act and FINRA rules. Frontier is designed and labelled as an **educational tool**: it charges nothing, has no affiliate or broker incentives, and every page carries the disclaimer that it is not advice.

Design choices that reduce harm: (1) the questionnaire separates **risk tolerance** from **risk capacity** and applies hard **caps** — an emergency fund, a horizon under three years, or "I'd sell everything in a crash" limit the profile regardless of other answers; (2) model portfolios are diversified, low-cost index ETFs with **no return forecasts**, so they can't encode last decade's winners; (3) every recommendation shows its downside first-class: worst drawdown in money, worst calendar year, the 10th-percentile outcome; (4) trend scores are explicitly described as descriptions of the past, not predictions; (5) crypto is flagged when above 10 % with plain guidance; (6) no dark patterns: nothing is sold, nothing is tracked.

What I'd add before a real launch: a legal review, suitability language reviewed by a registered adviser, clearer data-licensing (Yahoo's terms), and accessibility testing with real users.

Likely follow-up: "Isn't showing a 'Strong' score on a stock an implicit buy signal?" → arguably, which is why the copy says "describes recent price behaviour… not a prediction" and why model portfolios never use it.

Q17 How do you test numerical code? Which tolerances did you choose and why? Give a real example where a test caught a bug.

What they're testing: Testing maturity for non-deterministic and floating-point code.

Three layers. **Golden tests** compare against independent reference implementations on byte-identical inputs; tolerances follow the algorithm: 1e-15 for closed forms (Ledoit–Wolf), 1e-12 to 1e-13 for deterministic algorithms (HRP, Black–Litterman), and for convex programs I compare the *objective* (ours ≥ reference − 1e-6) rather than weights, because interior-point solvers stop at ~1e-8 duality gaps and degenerate problems have multiple optimal weight vectors. **Property tests** check invariants: projections are feasible, idempotent and satisfy KKT (shared shift on free coordinates) over 200 random cases; the frontier is monotone; the tangency beats every random portfolio; risk contributions match arbitrary budgets. **Analytic tests** compare to closed forms: zero-volatility Monte Carlo vs the SIP formula; the lognormal median CAGR ≈ $e^{(\mu-\sigma^2/2)} - 1$ with 20,000 paths; HMM parameter recovery on data with known regimes.

Real catches: (1) the OLS test failed by 2e-9 against NumPy's `lstsq`. A “harmless” default ridge of 1e-12·trace(X^TX) was biasing coefficients of small-scale regressors; I removed it and rely on Cholesky jitter only for true collinearity. (2) The ERC test “failed” by 5e-6. Measuring risk contributions showed ours equal to 1e-10 and Clarabel's only to 8e-6 — the reference was the less accurate one, so the test now asserts the defining property and compares weights at solver tolerance. (3) The Monte Carlo drift bias in Q10. (4) The group-constraint performance cliff in Q6 surfaced as a 4.2-second test.

Likely follow-up: “How do you test randomness?” → every stochastic routine takes a seed (sfc32 PRNG); tests assert reproducibility and statistical properties with tolerances sized to the Monte Carlo error.

Q18 What is look-ahead bias? Name every place it could creep into Frontier — including the ones you didn't fully fix.

What they're testing: Intellectual honesty and research rigour. Volunteering limitations scores highly.

Look-ahead bias is using information in a decision that wasn't available at the time, which makes backtests look better than reality. Audit:

- **Regimes** — avoided: live signals use filtered, not smoothed, HMM probabilities.
- **Walk-forward tests** — avoided: each rebalance uses only the trailing 252 days, prices at the rebalance close, and costs on turnover.
- **Model portfolio backtests** — *partially present*: sleeve weights come from covariances of the last five years and are then applied to the whole history. ERC uses only covariances (more stable than returns), but the reported backtest is still mildly in-sample. The honest fix is to compute each year's weights from data before that year; I'd present that version first.
- **Survivorship bias** — *present*: the stock universe is today's largest companies, so their 10-year histories are winners by construction. The app focuses beginners on index ETFs, and the Learn page explains it, but single-stock backtests overstate.
- **Adjusted prices** — subtle: Yahoo's adjusted closes are rewritten after each dividend; every run refetches full history, so the data is internally consistent, but historical values change slightly over time.
- **Proxies in stress tests** — disclosed: before an ETF existed, its index proxy is used, and the UI reports how much of the portfolio was covered by proxies.
- **Signal ranks** — avoided: peer-group percentiles use only current cross-sectional data.

Likely follow-up: “How big is survivorship bias?” → studies put it at roughly 1–2 % per year for US equity fund databases; for a mega-cap list chosen today it can be much larger.

Q19 Security review: there is no backend. What can still go wrong?

What they're testing: Threat modelling for serverless and static apps, supply chain and CI.

- **XSS via share links**: `#p=…` decodes attacker-controlled JSON. It is decompressed, parsed, type-checked (currency enum, finite positive weights, ≤ 40 holdings, name truncated) and

rendered only through React's escaping; there is no `dangerouslySetInnerHTML`. Unknown instrument IDs are ignored.

- **Third-party data**: WebSocket frames and market data are parsed defensively and rendered as text; a compromised feed could show wrong prices but not execute code.
- **Supply chain** — the biggest real risk: a malicious npm update would run in CI with a token and ship to users. Mitigations: lockfile + `npm ci`, Dependabot PRs reviewed by CI, CodeQL, major-pinned first-party Actions. Next steps: pin Actions to commit SHAs, enable npm provenance checks, add an SRI/CSP policy.
- **CI permissions**: `GITHUB_TOKEN` is least-privilege per workflow (deploy: `pages` + `id-token`; screenshots: `contents: write`; keepalive: `actions: write`). No long-lived secrets exist to steal.
- **Data poisoning**: if upstream data were manipulated, the validation gate catches structural anomalies, not plausible lies — cross-source checks would be the next layer.
- **Headers**: GitHub Pages can't set CSP, HSTS-preload or frame-ancestors headers. A `<meta http-equiv="Content-Security-Policy">` covers most of CSP (not frame-ancestors) — a listed gap.
- **Local data**: localStorage is readable by any script on the origin — another reason the dependency surface matters — but it holds only the user's own portfolios.

Likely follow-up: "Would you sign builds?" → Pages deployments use OIDC (`id-token: write`); adding artifact attestations is a one-line addition.

Q20 If you had two more weeks, what would you change first? What technical debt are you carrying?

What they're testing: Self-awareness and prioritisation. Strong candidates rank by user impact and risk.

1. **Data-vendor risk**: add a licensed end-of-day source behind `sources/` and use Yahoo as fallback, plus cross-source validation on core instruments.
2. **Point-in-time model backtests**: recompute sleeve weights each year from prior data, removing the remaining in-sample bias in the Advisor's statistics.
3. **Incremental ingestion** with dividend-revision detection, cutting upstream load by ~95 % and enabling more instruments.
4. **Factor model covariance** ($\Sigma = BFB^T + D$) for larger universes and better conditioning.
5. **Tax & cost realism**: tax-lot tracking, Indian LTCG/STCG and US capital-gains rules in the rebalancer; turnover-penalised optimisation.
6. **Front-end tests**: component tests for charts and visual regression snapshots; today the UI is covered by E2E only.
7. **Hardening**: CSP meta tag, SHA-pinned Actions, artifact attestations.
8. **Product**: optional encrypted sync, alerts, i18n (Hindi), and moderated user testing with real beginners.

Debt I'm consciously carrying: the unofficial data source; weekday forward-fill on holidays; overlapping asset-class constraints use the slower Dykstra path; the Market Mood index is a heuristic composite (transparent, but not validated against future returns).

Likely follow-up: “Which of these would you do first if you could only pick one?” → the data vendor: every feature depends on it and it is the only single point of failure.

7 Rapid-fire round, numbers to memorise, honest limitations

7.1 Rapid-fire questions (answer in one breath)

Question	Answer
Why weekly returns for beta?	Asynchronous closes (NSE vs NYSE) bias daily betas towards zero; weekly sampling dilutes the one-day offset.
Sharpe vs Sortino?	Sortino divides by downside deviation (returns below the risk-free rate), so upside volatility isn't penalised.
What is effective number of bets?	exp(entropy) of risk contributions of uncorrelated principal portfolios (Meucci 2009): 1 = one hidden bet, n = n independent bets.
Why is CAGR ≠ average return?	Compounding: +50 % then −50 % is −25 %. CAGR ≈ arithmetic mean − $\sigma^2/2$.
Why 6 significant digits in history files?	Rounding error < 5e-7, far below daily-return noise for even the calmest asset; with delta-encoded dates files compress to ~19 KB.
Why FNV-1a for content hashes?	Cache keys, not security: fast, tiny, deterministic. Collisions only cost a cache miss window.
Why cron at minute 17?	GitHub's scheduler is most congested at :00; off-peak minutes start more reliably.
What is the keepalive for?	GitHub disables schedules in public repos after 60 days without activity; re-enabling via the API resets the clock.
Why deterministic NLG instead of an LLM?	Free, instant, offline, reproducible, unit-testable — and it can't hallucinate a number.
Why separate tolerance and capacity?	Feeling brave doesn't make a two-year horizon long; regulators require suitability on both.
How are model portfolios built?	Profile sets stocks/stabilisers/gold; ERC inside each sleeve (Ledoit–Wolf, 5y weekly); cash takes half of a mixed sleeve; monthly rebalanced backtest with 5 bp costs.
Forward return assumption?	$\mu = ½\cdot\text{historical} + ½\cdot(r_f + 0.3\sigma)$: shrink halfway to a Sharpe-0.3 prior so a lucky decade isn't extrapolated.
How do deep links work on Pages?	post-build copies index.html into a folder per route and per asset; 404.html covers the rest.
What does the gate check?	zod schema, core IDs, ≥ 85 % coverage, SPY ≤ 6 days old, finite positive prices, ≥ 6 model portfolios.
Why a validated colour palette?	~8 % of men have red-green colour-vision deficiency; returns use blue ↔ red diverging scales and every signal has an icon + label.

7.2 Numbers to memorise

100 instruments (98 fetched + 2 cash)	**hourly** weekday refresh; 4-hourly weekends	**113** tests: 87 unit/golden + 26 E2E
1e-15 Ledoit–Wolf agreement with scikit-learn	**9 ms** max-Sharpe (6 assets), was 52 ms	**~220 KB** gzip initial JS+CSS; worker 24 KB
86 KB gzip snapshot for all dashboards	**85 %** minimum coverage to publish	**$0** monthly running cost

7.3 Honest limitations (say them before you're asked)

- Market data comes from Yahoo's unofficial endpoints; for production I'd license a vendor.
- Stock universe has survivorship bias; model-portfolio backtests are mildly in-sample (Q18).
- Stocks refresh hourly, not in real time; only crypto streams live.
- Holiday forward-fill slightly understates multi-market volatility.
- Taxes, lot-level cost basis and bid-ask spreads are not modelled (flat bp costs only).
- Market Mood is a transparent heuristic, not a validated predictor.
- No accounts or sync; clearing browser storage clears portfolios (share links are the backup).

> **Closing line for interviews**
> “The interesting part wasn't any single algorithm — it was making a system that is cheap enough to run forever, careful enough that a beginner can trust its numbers, and honest about what it doesn't know. Every number on the screen is reproducible, tested against an independent reference, and explained in a sentence.”